

TESI FINALE DI PROGETTAZIONE DI SISTEMI OPERATIVI

## Programmazione concorrente nel Linguaggio Java

Approfondimento sull'utilizzo dei *Thread* in Java

Studente:

**Tommaso Cosimi**

Matricola 191739

Docente:

**Prof.ssa Letizia Leonardi**

Codocente:

**PhD Silvia Cascianelli**



## Sommario

Nel corso della Storia dell'Informatica, l'avanzamento tecnico e tecnologico verificatosi nell'ambito delle Architetture dei Calcolatori e dei Sistemi Operativi ha permesso grandi passi in avanti nelle dinamiche di utilizzo degli Elaboratori Elettronici. Si è assistito al passaggio dall'esecuzione sequenziale delle operazioni ad un'esecuzione parallela delle stesse, il tutto all'interno di un singolo Elaboratore.

L'opportunità di avere esperienze utente multiprogrammate è stata resa possibile, oltre che dagli Elaboratori non sequenziali, anche da Linguaggi di Programmazione anch'essi non sequenziali che permettano l'esecuzione concorrente di più processi. Tale flessibilità ha permesso un grado di utilizzo delle risorse nettamente superiore al passato, permettendo prestazioni superiori a parità di risorse.

Lo scopo di questo elaborato sarà quello di approfondire l'argomento della Programmazione Concorrente nel Linguaggio Java, riportando allo stesso tempo esempi pratici di utilizzo.



# Indice

|          |                                          |           |
|----------|------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                      | <b>11</b> |
| 1.1      | Algoritmo, Programma, Processo . . . . . | 11        |



## Elenco delle figure



## Elenco delle tabelle



# Elenco degli algoritmi

|   |                              |    |
|---|------------------------------|----|
| 1 | <i>Bubble Sort</i> . . . . . | 12 |
|---|------------------------------|----|



# Capitolo 1

## Introduzione

Il seguente Capitolo fornirà un'introduzione teorica al concetto di esecuzione non sequenziale di un processo, ponendo enfasi sui vantaggi portati e sulle nuove necessità richieste da tale implementazione.

### 1.1 Algoritmo, Programma, Processo

Nella definizione della singola unità di esecuzione si ritiene opportuno effettuare una leggera digressione.

#### Algoritmo

Un Algoritmo è un procedimento puramente logico studiato appositamente per la risoluzione di uno specifico problema. In letteratura sono rintracciabili moltissimi Algoritmi, per brevità vengono nominati a titolo di esempio Algoritmi di Ordinamento come il *Bubble Sort*[1] - di cui viene fornito lo pseudocodice a seguire - od il *Radix Sort*[2], e Algoritmi di Ricerca del Cammino Minimo in un Grafo come l'*Algoritmo di Dijkstra*[3] o l'*Algoritmo di Bellman-Ford*[4].

---

**Algoritmo 1:** Ordinamento per confronti “*Bubble Sort*”

---

```

Dati: A;                // Vettore di numeri interi non ordinati
1 partizioneNonOrdinata  $\leftarrow$  A.length;
2 finché partizioneNonOrdinata > 1 fai
3   per i = 2 to k fai
4     se A[i] < A[i - 1] allora
5       |   Scambia A[i] ed A[i - 1];
6     fine se
7   fine per
8   partizioneNonOrdinata  $\leftarrow$  (ultimoScambio - 1)
9 fine finché
10 ritorna A;            // Vettore di numeri interi ordinati

```

---

**Programma**

Un Programma è la descrizione in codice di un Algoritmo in uno specifico Linguaggio di Programmazione in maniera tale da permetterne l'esecuzione all'interno di un Calcolatore, è un'entità statica. Si riporta di seguito un'esempio di Programma che descriva l'Algoritmo di Ordinamento per confronti “*Bubble Sort*” implementato tramite il Linguaggio di Programmazione Java.

```

1 public class BubbleSort {
2
3     public static void BubbleSort(int[] A) {
4
5         // Definizione della dimensione della partizione del
6         //  $\hookrightarrow$  vettore non ordinata
7         int unorderedPartition = A.length;
8
9         // Ordinamento
10        while(unorderedPartition > 1) {
11            // Scorro la parte non ordinata
12            for(int i=2; i<unorderedPartition; i++) {
13                // Se non e' ordinato, scambia
14                if(A[i] < A[i-1]) {
15                    int buf = A[i];
16                    A[i] = A[i-1];
17                    A[i-1] = buf;
18                }
19            }
20            unorderedPartition--;
21        }
22    }
23 }

```

```
16         A[i-1] = buf;
17     }
18 }
19     // Dopo lo scambio, riduco la dimensione della
        ↪ partizione del vettore non ordinata
20     unorderedPartition = unorderedPartition - 1;
21 }
22
23 }
24
25 }
```

## Processo

Un Processo è la sequenza di operazioni eseguite da un Elaboratore che opera sotto il controllo di un Programma, è un'entità dinamica: in maniera intuitiva è possibile affermare che in linea generale un Processo è un Programma in esecuzione. Di seguito si riporta un esempio di esecuzione sequenziale di due Processi che eseguono lo stesso Programma, ovvero *Bubble Sort*.

```
1 [tcosimi@VivoBook-Arch]:$ java BubbleSort 1 2 3 6 5 2
2 Il vettore non ordinato e': [1, 2, 3, 6, 5, 2]
3 Il vettore ordinato e': [1, 2, 2, 3, 5, 6]
4 [tcosimi@VivoBook-Arch]:$ java BubbleSort 1 9 8 3 6 7
5 Il vettore non ordinato e': [1, 9, 8, 3, 6, 7]
6 Il vettore ordinato e': [1, 3, 6, 7, 8, 9]
```



# Bibliografia

- [1] Wikipedia. Bubble Sort, Giugno 2023. Articolo disponibile all'url: [https://it.wikipedia.org/wiki/Bubble\\_sort](https://it.wikipedia.org/wiki/Bubble_sort). Pagina visitata il 29 ottobre 2023.
- [2] Wikipedia. Radix Sort, Maggio 2023. Articolo disponibile all'url: [https://it.wikipedia.org/wiki/Radix\\_sort](https://it.wikipedia.org/wiki/Radix_sort). Pagina visitata il 29 ottobre 2023.
- [3] Wikipedia. Algoritmo di Dijkstra, Aprile 2023. Articolo disponibile all'url: [https://it.wikipedia.org/wiki/Algoritmo\\_di\\_Dijkstra](https://it.wikipedia.org/wiki/Algoritmo_di_Dijkstra). Pagina visitata il 29 ottobre 2023.
- [4] Wikipedia. Algoritmo di Bellman-Ford, Maggio 2023. Articolo disponibile all'url: [https://it.wikipedia.org/wiki/Algoritmo\\_di\\_Bellman-Ford](https://it.wikipedia.org/wiki/Algoritmo_di_Bellman-Ford). Pagina visitata il 29 ottobre 2023.